

One False Sum:

A complete, formally verified characterization of solvability
in a splitting–merging game with a single arithmetic lie

July 2026

Abstract

We study a one-player game on finite multisets of positive integers in which a ball of value n may be split into $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, and two balls may be merged into their sum—except that the arithmetic is governed by a single *false sum* $a + b = c$: the ball c splits only into a and b , and the pair $\{a, b\}$ merges only into c . Writing $g = |a + b - c|$ and $M = \max(a + b, c) + 1$, we prove that for *every* false sum (a, b, c) with $a, b, c \geq 1$ and all $s, t \geq M$, the single ball s can be transformed into the single ball t if and only if $g \mid t - s$; moreover the threshold M cannot be lowered to $M - 1$ in general. The sufficiency proof is a taxonomy: a generic “all-ones hub” handles most parameters, but it provably breaks down on several degenerate families—the *traps*, in which some ball values can never be reduced to units—each of which requires its own construction, including one eleven-move gadget for the single most pathological shape $\langle c, 2c, c \rangle$. A dichotomy lemma (every value $v \geq c$ is either of the trap form $2^k c$ or “scatterable”) glues the taxonomy into a complete case analysis. All results, from the move semantics up to the final characterization, are formally verified in Lean 4 (core library only), in roughly 7,500 lines comprising 328 theorems with no unproven assumptions.

1 Introduction

Imagine a puzzle in which you are handed a single ball labelled with a positive integer and asked to produce a single ball with a different label. Two operations are available: *splitting* a ball into two balls whose labels are the (integer) halves of the original, and *merging* two balls into one whose label is their sum. Splitting and merging are exact inverses of one another in spirit, and with true arithmetic the game is uninteresting: the total is invariant, so a single ball s can become a single ball t only when $s = t$.

The game becomes interesting when the arithmetic is wrong. Fix a single *false sum*: an assertion $a + b = c$ where possibly $a + b \neq c$. The player is committed to this belief.¹ Consequently the ball c no longer splits into its halves—it splits into a and b —and the pair $\{a, b\}$ no longer merges to $a + b$ but to c . Every false move changes the total by $\pm(a + b - c)$, so the total is no longer invariant and genuine transformations $s \rightsquigarrow t$ with $s \neq t$ become possible. Two natural questions arise:

¹The game we formalize, *Ya Stupid*, takes its name from a 2014 viral video in which a child, asked “what’s nine plus ten?”, confidently answers “twenty-one.” The false sum $9 + 10 = 21$ is accordingly called the *Classic* configuration below.

(Q1) Exactly which pairs (s, t) are solvable?

(Q2) Is there a threshold above which the answer depends only on the obvious congruence obstruction?

Our main result answers both completely, for *every* false sum.

Theorem 1.1 (Main; formally verified). *Let $a, b, c \geq 1$, let $g = |a + b - c|$ and $M = \max(a + b, c) + 1$. For all $s, t \geq M$,*

$$\{s\} \longrightarrow^* \{t\} \iff g \mid t - s.$$

(When $g = 0$, i.e. the “false” sum is true, the right-hand side reads $t = s$.) Moreover, the threshold M is optimal: it cannot be lowered to $M - 1$. For the Classic configuration $9 + 10 = 21$ one has $M = 22$, the pair $(21, 23) = (M - 1, M + 1)$ satisfies the congruence, and yet $\{21\} \not\longrightarrow^* \{23\}$.

The forward implication is an invariant argument and holds far more generally (for any finite set of simultaneous false sums, with g their gcd); the content of the theorem is the converse, *sufficiency*.

What makes sufficiency interesting is that the natural proof strategy is *false*. The natural strategy—call it the *all-ones hub*—is to scatter the ball s into s unit balls, adjust the size of the pile of units by firing false moves, and then rebuild the target t . For many parameters this works, and §5 makes it precise. But the game contains genuine *traps*: configurations in which certain balls can never be reduced to units at all. The simplest is $2 + 2 = 2$, where a lone ball 2 only ever produces multisets of 2’s (its only split is the false one, $2 \rightarrow \{2, 2\}$, and two 2’s only merge falsely back to a 2). More elaborately, when both legs a, b are of the form $c \cdot 2^k$ (as in $6 + 6 = 3$ or $3 + 6 = 3$), every route from a large ball down towards units jams on *locked* copies of c , which refuse to halve. Each trap family needs its own escape:

- for the traps with $c \geq 3$, a hub built on *copies of c* rather than units, driven by a peeling recursion (§7);
- for the single most pathological shape $\langle c, 2c, c \rangle$ (“ $c + 2c = c$ ”), where even the peeling recursion jams at one specific ball value, a bespoke eleven-move excursion that temporarily doubles the total (§7.1);
- for $c = 2$, a hub built on copies of 2, with odd values manufactured by *splitting* even balls rather than assembling units (§8);
- for the doubly degenerate family $1 + c = c$, in which the naïve peel-off-a-unit move is unavailable in principle, a lose-exactly-one recursion (§7.2);
- for $c = 1$, an unobstructed hub (§8).

A short *dichotomy lemma* (Lemma 6.2: every $v \geq c$ is either exactly $2^k c$, or “scatterable”) proves that this taxonomy is exhaustive, and a final assembly theorem dispatches every (a, b, c) to its family.

Formal verification. Every result in this paper—the move semantics, the invariant, the sharpness witness, each family construction, the dichotomy, and the final characterization—has been formally verified in the Lean 4 proof assistant [1], using only Lean’s core library (no Mathlib). The development is a single file of 7,462 lines containing 328 theorems; it compiles in under a minute and depends only on the three standard axioms of Lean’s classical logic (propositional extensionality, choice, quotient soundness)—in particular there are no unproven assumptions (**sorry-free**). The statements quoted in this paper correspond to the Lean theorems named in §10. In the mathematical sections below we therefore favour explaining the *constructions* over spelling out routine bookkeeping, which the formalization pins down exactly.

A caveat on independent evidence. During the development, bounded breadth-first searches over the state space were used to hunt for counterexamples and to discover candidate move sequences. No counterexample to Theorem 1.1 was ever found; all search results were subsequently superseded by machine-checked proofs. Claims below of the form “ X is unreachable” are proved (Proposition 3.2); claims of the form “the naïve strategy fails” are proved where stated and otherwise serve only as motivation.

2 The game

Definition 2.1 (States and moves). Fix $a, b, c \geq 1$ (the *false sum*, written $\langle a, b, c \rangle$). A *state* is a finite multiset of positive integers, called *balls*. The legal moves are:

- **normal split:** replace one ball n with $n \geq 2$ and $n \neq c$ by the two balls $\lfloor n/2 \rfloor, \lceil n/2 \rceil$;
- **false split:** replace one ball of value c by the two balls a, b ;
- **normal merge:** replace two balls x, y with $\{x, y\} \neq \{a, b\}$ (as multisets) by the ball $x + y$;
- **false merge:** replace the two balls a, b by the ball c .

We write $S \longrightarrow T$ if T is obtained from S by one move, and $\longrightarrow^*$ for the reflexive–transitive closure.

The player’s false belief is *forced*: the ball c cannot be halved (its only split is the false one), and the pair $\{a, b\}$ cannot merge to $a + b$ (its only merge is the false one). Note the model is manifestly symmetric in $a \leftrightarrow b$, a fact we prove and use (Lemma 9.1).

Definition 2.2 (Parameters). Set $\delta = a + b - c \in \mathbb{Z}$, $g = |\delta|$, and $M = \max(a + b, c) + 1$.

Example 2.3. Classic ($9 + 10 = 21$): $g = 2$, $M = 22$. The false split lowers the total by 2; the false merge raises it by 2. For $6 + 6 = 3$: $g = 9$, $M = 13$. For $1 + 3 = 3$: $g = 1$, $M = 5$ —so Theorem 1.1 asserts that above 5 *every* pair is solvable. Table 1 lists more.

false sum	g	M	character
$9 + 10 = 21$	2	22	Classic; deficient ($a + b < c$)
$3 + 3 = 5$	1	7	excessive, both legs $< c$
$8 + 9 = 4$	13	18	one leg of the form $c \cdot 2^k$
$15 + 15 = 4$	26	31	large legs, not powers
$6 + 6 = 3$	9	13	trap: $a = b = 2c$
$3 + 6 = 3$	6	10	the pathological $\langle c, 2c, c \rangle$
$2 + 2 = 2$	2	5	trap: $a = b = c$
$4 + 6 = 2$	8	11	$c = 2$, both legs even
$1 + 3 = 3$	1	5	doubly degenerate $\langle 1, c, c \rangle$
$2 + 3 = 1$	4	6	$c = 1$

Table 1: Sample false sums with their inaccuracy g and threshold M . Each is solvable for all $s, t \geq M$ with $g \mid t - s$ (Theorem 1.1).

3 Necessity and sharpness

Proposition 3.1 (Necessity). *If $\{s\} \longrightarrow^* \{t\}$ then $g \mid t - s$.*

Proof. Normal moves preserve the total of a state; the false split changes it by $(a + b) - c = \delta$ and the false merge by $-\delta$. Hence the total is constant modulo g along any play, and the totals of the two singleton states are s and t . $\square$

The formalization proves this for an arbitrary finite *set* of simultaneous false sums, with g the gcd of the individual inaccuracies; only sufficiency is specific to a single lie.

Proposition 3.2 (Sharpness at Classic). *For $\langle 9, 10, 21 \rangle$ one has $M = 22$: $\{21\} \not\longrightarrow^* \{23\}$, although $21, 23 \geq M - 1 = 21$ and $2 \mid 23 - 21$.*

Proof. Call a state *good* if all its balls are positive and either its total is 19 or it is exactly $\{21\}$. We claim goodness is preserved by every move; since $\{23\}$ is not good, this proves the proposition. From $\{21\}$, the only available move is the false split ($21 = c$ cannot halve; nothing can merge), yielding $\{9, 10\}$ of total 19: good. From a good state of total 19: a false split would require a ball $21 > 19$, impossible; a false merge requires balls 9 and 10, and since $9 + 10 = 19$ is the whole total, the state is exactly $\{9, 10\}$ and the result is exactly $\{21\}$: good. All other moves are normal and preserve the total 19. $\square$

Thus the guarantee of Theorem 1.1 fails at $M - 1 = \max(a + b, c)$ itself, so M is the optimal uniform threshold expressible in terms of $\max(a + b, c)$. (For particular false sums a smaller threshold may hold; we do not pursue the per-configuration optimum here.)

4 Sufficiency I: reduction to two pumps

Everything below serves the converse direction. The first step is a routine reduction.

Lemma 4.1 (Pump reduction). *Suppose the two pumps hold for a given $\langle a, b, c \rangle$:*

$$(climb) \quad \forall n \geq M : \{n\} \longrightarrow^* \{n + g\}, \quad (descend) \quad \forall n \geq M : \{n + g\} \longrightarrow^* \{n\}.$$

Then $\{s\} \longrightarrow^* \{t\}$ for all $s, t \geq M$ with $g \mid t - s$.

Proof. Iterate the appropriate pump $|t - s|/g$ times; every intermediate value stays $\geq M$. $\square$

Note that reachability is *not* symmetric move-by-move (a normal merge $\{3, 7\} \rightarrow 10$ reverses only via a longer route), so climb and descend genuinely require separate constructions. The remainder of the paper discharges the two pumps for every (a, b, c) , by cases.

5 Sufficiency II: the all-ones hub

Throughout this section, “units” are balls of value 1. The hub strategy for $\{n\} \longrightarrow^* \{n \pm g\}$ is:

1. *scatter* $\{n\}$ into the pile of units 1^n ;
2. change the pile size by $\pm g$ using one false move (*gain*: create a ball c , false-split it into $\{a, b\}$, re-scatter, for $a + b > c$; *lose*: assemble a and b , false-merge to c , re-scatter; roles swap when $a + b < c$);
3. *rebuild* the single ball from the pile.

Each step must dodge the two special values: a ball c is *locked* (it cannot halve), and the pair $\{a, b\}$ is *poisoned* (it merges only falsely). The precise dodges depend on the parameters.

5.1 The deficient case $a + b < c$

Here $a, b < c$, and every value below c is *free*: its halves stay below c , so it scatters to units by repeated normal splits, and units re-assemble into any value $k \leq \max(a, b)$ (merging a unit onto $j < k$ can only form the poisoned pair if $\{1, j\} = \{a, b\}$, which forces $\max(a, b) = j < k \leq \max(a, b)$ —absurd). The climb pump on the window $n \in [c + 1, 2c - 2]$ is then: split n into two sub- c halves, scatter both, assemble a and b , false-merge to c ($+g$), and reel the remaining units onto the c (safe: $c > \max(a, b)$, so $\{x, 1\}$ with $x \geq c$ is never poisoned). The three boundary values $n \in \{2c - 1, 2c, 2c + 1\}$, whose halves collide with the locked c , are handled by short explicit routes, and a halving recursion (split, pump the half, re-merge) reduces every $n \geq M$ to the window. The descend pump is dual (false-split a freshly built c).

Two edges of this case need extra care, both caused by unit legs. If $a = 1$ the pair $\{1, b\}$ is poisoned, so re-assembly must *skip* the value $b + 1$: build b and a separate 2, merge them to $b + 2$ (legal: $\{b, 2\}$ is never $\{1, b\}$), and continue. If $a = b = 1$, units cannot merge *at all* ($\{1, 1\}$ is the false pair, merging to c !); the construction instead maintains *2-seeds*—balls of value 2 produced by splitting odd values—and grows from those.

5.2 The excessive case with small legs

For $a + b > c$ and both legs $< c$ the same hub runs with the roles of the false moves exchanged (the false *split* now gains g). This case includes every “cluster structure” of the boundary values and is fully generic; the formal statement covers all $2 \leq a, b < c < a + b$.

When exactly one leg is small ($2 \leq a < c \leq b$, say), scattering the big leg may jam on locked c ’s, and the following device—used pervasively from here on—resolves it.

Lemma 5.1 (Reservoir bumping). *Let $c \geq 3$ and suppose $\{1, c\} \neq \{a, b\}$. From any state containing a ball $v \geq 1$ and $K \geq 1$ units, one can reach the state with $v + K$ units.*

Proof sketch. Halve v repeatedly. Whenever a locked c appears, *bump* it: merge one unit onto it (legal by hypothesis), obtaining $c + 1$, which is unlocked and halves into two values $\leq \lceil (c + 1)/2 \rceil \leq c - 1$, both free to continue. The unit invested is recovered because all fragments eventually reach 1. A strong induction on the multiset of values makes this terminate. $\square$

Thus a single unit “unlocks” everything—the problem is only ever *obtaining the first units*, which is exactly what the trap families prevent.

6 Scatterability, and the trap dichotomy

For $a + b > c$ with both legs $\geq c$ we need to know which single balls can produce units on their own.

Definition 6.1 (Scatterable values). For $c \geq 3$, let $\text{Scat}_c \subseteq \mathbb{N}$ be the least set such that: $v \in \text{Scat}_c$ whenever $c < v < 2c$; and $v \in \text{Scat}_c$ whenever $v \geq 2c$ and $\lfloor v/2 \rfloor \in \text{Scat}_c$ or $\lceil v/2 \rceil \in \text{Scat}_c$.

A value $c < v < 2c$ splits into a half $< c$ (which scatters freely into a reservoir of units) plus a remainder which is then bumped (Lemma 5.1); a larger scatterable value halves towards the window. Hence, under the safety hypothesis of Lemma 5.1, every $v \in \text{Scat}_c$ satisfies $\{v\} \rightarrow^* 1^v$ on its own. The values *not* in Scat_c are precisely characterized:

Lemma 6.2 (Dichotomy). *Let $c \geq 3$. Every $v \geq c$ satisfies: $v = 2^k c$ for some $k \geq 0$, or $v \in \text{Scat}_c$.*

Proof. Strong induction on v . If $c \leq v < 2c$: either $v = c = 2^0 c$, or v is in the window. If $v \geq 2c$ is even, its two halves equal $v/2 \geq c$; by induction $v/2$ is a power multiple (making v one) or scatterable (making v scatterable). If $v \geq 2c$ is odd, its halves are consecutive integers $f, f + 1 \geq c$; they cannot *both* be of the form $2^i c$, since $2^j c - 2^i c = 1$ would force $c \mid 1$. Apply induction to a half that is not a power multiple: it is scatterable, hence so is v . $\square$

Consequently, if at least one leg is not of the form $2^k c$, that leg scatters on its own; a false split $c \rightarrow \{a, b\}$ then converts any locked c into a scatterable pile, every value scatters, and the hub closes the configuration. (This subsumes, in one stroke, all “bands” of leg sizes.) What remains is exactly:

both legs of the form $2^k c$ and the small targets $c \in \{1, 2\}$ and unit legs.

7 Sufficiency III: the traps

Let now $a = 2^p c$ and $b = 2^q c$, $c \geq 3$. These are genuine traps for the hub: for instance in $\langle 2, 2, 2 \rangle$ a lone ball 2 generates only multisets of 2’s (its only split is $2 \rightarrow \{2, 2\}$ and two 2’s merge only falsely back to 2), and the analogous c -multiple phenomena occur for all these families. We do not need any unreachability statement, however; we simply replace the commodity: *hub on copies of c instead of units*.

Lemma 7.1 (Peeling). *Let $a = 2^p c$, $b = 2^q c$, $c \geq 2$, and suppose $\{a, b\} \neq \{c, 2c\}$. Then for every $v \geq c + 1$: $\{v\} \longrightarrow^* \{c, v - c\}$.*

Proof sketch. Strong induction. For $v \leq 2c - 2$ both halves of v are $< c$, hence free: scatter everything to units and re-assemble c and $v - c$ (all merges among sub- c values and units are unpoisoned, since $a, b \geq c$). The boundary values $2c - 1, 2c, 2c + 1$ split directly into $\{c - 1, c\}, \{c, c\}, \{c, c + 1\}$. For $v \geq 2c + 2$: split v , peel a c off the lower half by induction, and re-merge the two leftovers $\lfloor v/2 \rfloor - c$ and $\lceil v/2 \rceil$. This merge is poisoned only if $\{\lfloor v/2 \rfloor - c, \lceil v/2 \rceil\} = \{a, b\}$, which forces $|a - b| \in \{c, c + 1\}$. For legs $2^p c \neq 2^q c$ the gap $|a - b|$ is a nonzero multiple of c , so only $|a - b| = c$ is possible, and $2^q - 2^p = 1$ then forces $(p, q) = (0, 1)$ (Lemma 9.2): that is, $\{a, b\} = \{c, 2c\}$, excluded by hypothesis. $\square$

Given peeling, the pumps are short. *Climb*: peel one c off $\{n\}$, false-split it into $\{a, b\} (+g)$, and merge everything back (both merges avoid the poisoned pair because the accumulating ball exceeds b). *Descend*: peel $2^p + 2^q$ copies of c off $\{n + g\}$, rebuild the two legs from the copies, false-merge $\{a, b\} \rightarrow c (-g)$, and merge back. Rebuilding a leg $2^k c$ from 2^k copies of c uses *binary doubling*, $\{x, x\} \rightarrow \{2x\}$, which can never hit the poisoned pair when $a \neq b$; when $a = b$ (so $p = q \geq 1$) one instead merges copies sequentially, $\{c, jc\} \rightarrow \{(j + 1)c\}$, unpoisoned since $jc < a$ and $c \neq a$. The diagonal $a = b = c$ is simplest of all: climb peels one c and false-splits it; descend peels two and false-merges them.

7.1 The pathological shape $\langle c, 2c, c \rangle$

One shape survives the analysis above: $a = c$, $b = 2c$ (“ $c + 2c = c$ ”, e.g. $3 + 6 = 3$), where the peeling recursion of Lemma 7.1 jams, at exactly one ball value. At $v = 4c$ the split gives $\{2c, 2c\}$, peeling a c off one half leaves $\{c, 2c\}$ —precisely the poisoned pair—and no local reordering avoids it. The escape is an eleven-move excursion that climbs to total $8c$, forges the otherwise unobtainable ball $3c$ by splitting a $6c$, and rides two false merges back down:

$\{4c\} \longrightarrow \{2c, 2c\}$	split $4c$	(total $4c$)
$\longrightarrow \{c, c, 2c\}$	split $2c$	
$\longrightarrow \{c, c, 2c, 2c\}$	false split $c \rightarrow \{c, 2c\}$	$(+g)$
$\longrightarrow \{c, c, 2c, 2c, 2c\}$	false split $c \rightarrow \{c, 2c\}$	$(+g)$ (total $8c$)
$\longrightarrow \{c, c, 2c, 4c\}$	merge $\{2c, 2c\}$	
$\longrightarrow \{c, c, 6c\}$	merge $\{2c, 4c\}$	
$\longrightarrow \{c, c, 3c, 3c\}$	split $6c$	
$\longrightarrow \{c, 3c, 4c\}$	merge $\{c, 3c\}$	
$\longrightarrow \{c, 2c, 2c, 3c\}$	split $4c$	
$\longrightarrow \{c, 2c, 3c\}$	false merge $\{c, 2c\} \rightarrow c$	$(-g)$
$\longrightarrow \{c, 3c\}$	false merge $\{c, 2c\} \rightarrow c$	$(-g)$ (total $4c$)

With this single gadget grafted in at $v = 4c$, the peeling recursion—and with it both pumps—goes through for $\langle c, 2c, c \rangle$, for every $c \geq 2$.

7.2 The doubly degenerate family $\langle 1, c, c \rangle$

The false sum $1 + c = c$ (any $c \geq 2$; $g = 1$) is a trap of a different kind: $a = 1$ makes the pair $\{1, c\}$ poisoned, so the ubiquitous “merge a unit onto a locked c ” bump of Lemma 5.1 is illegal,

and the ball c regenerates under its own false split ($c \rightarrow \{1, c\}$). The climb pump is still easy (peel a c —Lemma 7.1 applies, its recursion never jams here—false-split it, and re-merge, for a net $+1$). The descend pump must *lose exactly one* from $\{n+1\}$, and the construction is a recursion we call **loseOne**: peel copies of c off the ball until the remainder drops below c , scatter the (free) remainder into units, ride *one* false merge $\{1, c\} \rightarrow c$ (erasing a single unit, -1), and re-assemble—the re-assembly dodging $\{1, c\}$ throughout because the accumulated c -copies merge with each other and with sub- c values only. One divisibility wrinkle remains: if $c \mid n+1$ the peeling bottoms out with no remainder at all (an all- c 's pile). The fix is to *overshoot*: climb once more to $n+2 \equiv 1 \pmod{c}$, peel down to the remainder $c+1$, scatter it, and ride *two* false merges. For $c=2$ this last route needs units from a pile of 2's, obtained not by false-splitting (which regenerates) but by the *odd-ball core*: merge three 2's into 6, split $6 \rightarrow \{3, 3\}$, and split each $3 \rightarrow \{1, 2\}$ —manufacturing two units by *normal* moves alone.

8 Sufficiency IV: the small targets $c=1$ and $c=2$

$c=1$. Nothing is ever locked: normal splitting requires $n \geq 2 \neq c$, so every ball halves freely, and only the poisoned pair $\{a, b\}$ needs dodging. The hub runs essentially unobstructed; the one point of care is that for $a=1$ the re-assembly must skip the poisoned $\{1, b\}$ exactly as in the deficient case (build $b-1$ and a spare 2, or two 2's when $b=2$).

$c=2$. Here units are a scarce resource: the ball 2 is locked (it false-splits into $\{a, b\}$), so an even ball can decompose into 2's but no further. The construction hubs on *copies of 2*, with a two-route peeling lemma ($\{v\} \rightarrow^* \{2, v-2\}$ for $v \geq 3$: when the standard recursion would produce the poisoned pair, peel off the *other* half, whose residue differs by at most one and is therefore safe), and rebuilds legs from 2's. Odd legs are manufactured by *splitting* an even ball ($\{2, 2, 2\} \rightarrow \{6\} \rightarrow \{3, 3\} \rightarrow \{1, 2, 3\}$, a gadget that also yields a spare unit and a spare 2, both of which are absorbed harmlessly). The analysis splits into the four parity classes of (a, b) , plus the degenerate configurations where a leg equals 2 (then $a=c$, and every $\{x, 2\}$ with $x=b$ is poisoned) or equals 1; each is closed by a variant of the same two devices. We spare the reader the case ledger—the formalization contains ten theorems whose union is: *every $\langle a, b, 2 \rangle$ with $a+b > 2$ satisfies both pumps*.

9 The assembly

Three glue facts combine the families into a single statement.

Lemma 9.1 (Swap). *Reachability, g , and M are invariant under $\langle a, b, c \rangle \mapsto \langle b, a, c \rangle$.*

Lemma 9.2 (Power gap). *If $2^q = 2^p + 1$ then $p=0$ and $q=1$. Consequently, for legs $a = 2^p c \neq b = 2^q c$: $b = a + c$ forces $\{a, b\} = \{c, 2c\}$, and $b = a + c + 1$ is impossible for $c \geq 2$ (it would give $c \mid 1$).*

Theorem 9.3 (Characterization; formally verified). *For all $a, b, c \geq 1$ and all $s, t \geq M$:*

$$\{s\} \rightarrow^* \{t\} \iff g \mid t - s.$$

Proof (assembly). Necessity is Proposition 3.1. For sufficiency, by Lemma 9.1 assume $a \leq b$ and dispatch: if $a + b = c$, then $g = 0$ and $t = s$. If $a + b < c$: the deficient hub (§5), in its three variants ($2 \leq a$; $a = 1 < b$; $a = b = 1$). If $a + b > c$: for $c = 1$ and $c = 2$, §8; for $c \geq 3$ with $a = 1$, either $b > c$ (unit-leg hub, a bumping variant) or $b = c$ (§7.2); for $2 \leq a, b < c$, the excessive hub; for $a < c \leq b$, the small-leg bootstrap; and for $c \leq a \leq b$, apply Lemma 6.2 to both legs: if either is scatterable, the hub closes it (§6); otherwise both are powers $2^p c, 2^q c$ and the trap constructions apply—the diagonal $p = q = 0$, the equal case $p = q \geq 1$, the shape $\{c, 2c\}$ (via §7.1, whose applicability is exactly delimited by Lemma 9.2), and the generic $p \neq q$ (Lemma 7.1). Every branch discharges both pumps, and Lemma 4.1 concludes. $\square$

10 The formalization

The entire development is verified in Lean 4 (v4.31.0) [1] using only the core library—no Mathlib—in a single file (`lean/YaStupid.lean`, 7,462 lines, 328 theorems) in the repository accompanying this paper.² States are lists of naturals quotiented by permutation at the level of the step relation; the four moves of Definition 2.1 are an inductive family `Local`, one step applies a `Local` move to a sub-multiset (`Step`), and `Reach` is its reflexive–transitive closure. The headline statements are:

paper	Lean theorem
Theorem 9.3 ($\Leftarrow$, all a, b, c)	<code>single_sufficiency_all</code>
Theorem 9.3 (iff form)	<code>single_characterization</code>
Proposition 3.1 (any # of lies)	<code>reach_congr</code>
Proposition 3.2	<code>classic_trap</code>
Lemma 4.1	<code>sufficiency_of_pumps</code>
Lemma 6.2	<code>pow_or_scot</code>
Lemma 7.1	<code>peelcG</code>
§7.1 gadget	<code>peel4c</code>
§7.2 recursion	<code>loseOne</code>
Lemma 9.1	<code>reach_swap, suff_swap</code>
Lemma 9.2	<code>pow2_eq_succ</code>

The file compiles in about 40 seconds and every theorem is checked (via `#print axioms`) to depend only on Lean’s three standard classical axioms `propext`, `Classical.choice`, `Quot.sound`; in particular the development is free of `sorry`. Beyond the master theorems, the file contains machine-checked closed-form corollaries for the concrete false sums of Table 1 (e.g. `solvable_6_6_3`: all $s, t \geq 13$ with $9 \mid t - s$ are interreachable under $6 + 6 = 3$).

Formalization was not an afterthought: several “obvious” claims failed under mechanization and reshaped the mathematics. The all-ones hub was first believed to work whenever the pumps could bridge g -gaps; the trap families refuted this. The peeling recursion was first stated without the $\{c, 2c\}$ exclusion; Lean’s unprovable goal at the re-merge step is what isolated the pathological shape of §7.1 and the exact arithmetic of Lemma 9.2.

²<https://github.com/emblaisdell/yastupid>

11 Concluding remarks

- **Multiple lies.** For a finite set of simultaneous false sums the congruence obstruction (with g the gcd of the inaccuracies) is proved in the same formalization, and we conjecture the analogue of Theorem 9.3 with $M = \max_i \max(a_i + b_i, c_i) + 1$. The interaction of several locked values and poisoned pairs makes the trap taxonomy considerably richer, and we leave it open.
- **Per-configuration thresholds.** $M = \max(a + b, c) + 1$ is optimal as a uniform function of $\max(a + b, c)$ (Proposition 3.2), but individual false sums can admit smaller thresholds; characterizing the exact minimum for each (a, b, c) is open.
- **Beyond singletons.** Our characterization concerns single-ball endpoints. General multiset-to-multiset reachability adds a second obstruction (the number of balls interacts with the totals) and appears substantially harder.

Acknowledgements

The move sequences underlying several constructions were first located by bounded breadth-first search before being generalized and verified; we thank the machine for its patience.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pp. 625–635. Springer, 2021.